

Foundations of Deep Learning — Practice Exam

Master's Degree in AI & Society (PSL-PSAI, Spring 2026)

Exam 0—Solutions

Solutions

Part 1—True / False

1.1—TRUE. The composition of any number of affine maps $W_L(\cdots W_2(W_1x + b_1) + b_2 \cdots) + b_L$ is itself an affine map. Adding depth with linear activations buys no additional expressive power—this is the core motivation for using non-linear activations.

1.2—TRUE. Under $t = y(\mathbf{x}, \mathbf{w}) + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, \sigma^2)$, the log-likelihood is:

$$\log p(t \mid \mathbf{x}; \mathbf{w}) = -\frac{1}{2\sigma^2}(y(\mathbf{x}, \mathbf{w}) - t)^2 + \text{const.}$$

Maximising this over $\mathbf{w}$ is identical to minimising SSE.

1.3—FALSE. Gradient descent (in any form: batch, mini-batch, or stochastic) is only guaranteed to converge to a **stationary point** of the loss, i.e. a point where the gradient is zero. For non-convex loss functions, as is typical in deep learning, stationary points include local minima and saddle points in addition to the global minimum. No first-order method (that is, only involving first derivatives) can guarantee reaching the global minimum regardless of the learning rate schedule.

1.4—FALSE. The two operations are not equivalent.

Shift: adding δ to all logits leaves the softmax unchanged because e^δ factors out of both numerator and denominator and cancels:

$$\text{softmax}(a_i + \delta) = \frac{e^{a_i + \delta}}{\sum_j e^{a_j + \delta}} = \frac{e^\delta e^{a_i}}{e^\delta \sum_j e^{a_j}} = \frac{e^{a_i}}{\sum_j e^{a_j}}.$$

Scale: multiplying by c gives $e^{ca_i} / \sum_j e^{ca_j}$, which does **not** simplify to the original. For $c > 1$ the distribution sharpens (peaks become more pronounced); for $c < 1$ it flattens towards uniform. The c does not cancel because it appears inside the exponent, not as a multiplicative prefactor.

1.5—FALSE. During training, batch normalisation uses mini-batch statistics (μ_B, σ_B^2) . During inference it uses running averages accumulated over training:

$$\bar{\mu}_i^{(\tau)} = \alpha \bar{\mu}_i^{(\tau-1)} + (1 - \alpha) \mu_i,$$

because test batches may be small or contain a single example, making mini-batch statistics unreliable or undefined.

1.6—TRUE. Parameter count: each filter has shape $M \times M \times C_{in}$, giving $C_{in}M^2$ weights, plus 1 bias per filter. With C_{out} filters the total is $(C_{in}M^2 + 1)C_{out}$, independent of $H \times W$ —this follows directly from weight sharing: the same filter slides over every spatial position.

Comparison to fully connected: a fully connected layer mapping an input of size $H \times W \times C_{in}$ to an output of size $H \times W \times C_{out}$ would require $(H \times W \times C_{in} + 1) \times (H \times W \times C_{out})$ parameters. Since $H, W > M$ in any practical setting, this is vastly larger. Weight sharing is therefore the key computational advantage of convolutional layers.

1.7—TRUE. He initialisation draws weights from $\mathcal{N}(0, 2/M^{(l-1)})$. It is derived by requiring the variance of the pre-activations to remain stable across layers at initialisation. The factor of 2 accounts for the fact that ReLU zeroes out roughly half of its inputs on average, which would otherwise halve the variance at each layer.

1.8—TRUE. Without positional encodings, self-attention is permutation-equivariant: permuting the input tokens produces the same permutation of output representations. Both sentences contain the same multiset of tokens, so each token’s representation depends only on *which* tokens are present, not on their order. A classifier using pooled representations would produce identical predictions for both sentences.

Part 2—Multiple Choice

2.1—Correct: A, D.

(A) **True.** Standard derivation: $\partial \mathcal{L} / \partial a_i = y_i - t_i$.

(B) **False.** Softmax outputs are always in $(0, 1)$ for finite logits, so $-\log y_{t^*} > 0$ always. The loss approaches zero asymptotically as one logit diverges to $+\infty$, but never reaches it.

(C) **False.** Cross-entropy corresponds to maximum likelihood under a **categorical** distribution, not a Gaussian. It is the MSE/SSE loss that arises from MLE under a Gaussian noise assumption.

(D) **True.** For $K = 2$ with logits a_1, a_2 , the softmax gives $y_1 = e^{a_1} / (e^{a_1} + e^{a_2}) = \sigma(a_1 - a_2)$, where σ is the sigmoid function. The cross-entropy on $(y_1, y_2 = 1 - y_1)$ is therefore identical to binary cross-entropy applied to a single sigmoid output $\sigma(a_1 - a_2)$.

(E) **False.** Independent sigmoids need not sum to 1 and allow multiple classes to be simultaneously probable. This is the correct setup for multi-label classification, not single-label.

2.2—Correct: B, C, D.

(A) **False.** In high-dimensional spaces, the opposite is true: saddle points are exponentially more common than local minima. A local minimum requires all eigenvalues of the Hessian to be positive—an increasingly unlikely condition as the number of parameters grows.

(B) **True.** At a saddle point, the gradient is zero but the Hessian has both positive and negative eigenvalues. The loss decreases along directions corresponding to negative eigenvalues and increases along directions corresponding to positive eigenvalues.

(C) **True.** The optimal learning rate for a soft direction (low curvature) is large, while the optimal learning rate for a stiff direction (high curvature) is small. A single fixed learning rate cannot satisfy both simultaneously, leading to slow convergence or instability.

(D) **True.** By dividing the gradient by a running estimate of its root mean square, RMSProp and Adam effectively normalise the step size in each direction, taking larger steps in soft directions and smaller steps in stiff ones.

(E) **False.** Momentum smooths the gradient signal over time by accumulating a velocity vector—it does not rescale gradients component-wise. Adaptive methods such as Adam specifically address the stiff/soft problem by normalising each parameter’s update independently. The two mechanisms are complementary, which is why Adam combines both.

2.3—Correct: A, B, E.

(A) **True.** Reverse-mode automatic differentiation computes all partial derivatives $\partial\mathcal{L}/\partial\theta_i$ in a single backward pass. The computational cost is comparable to one forward pass—typically a factor of 2 to 3 overhead in practice.

(B) **True.** Backpropagation requires a well-defined topological ordering to propagate error signals; cycles would create undefined recursive dependencies.

(C) **False.** By the chain rule, the gradient at an early layer is the product of Jacobians through all subsequent layers:

$$\delta_j^{(l)} = h'(a_j^{(l)}) \sum_k w_{kj}^{(l+1)} \delta_k^{(l+1)}.$$

It explicitly depends on the weights and activations of all later layers through the recursion.

(D) **False.** The error signal explicitly involves the derivative of the activation function (see formula above). The choice of activation function directly affects the backward pass.

(E) **True.** The error signal requires the derivative of the activation function $h'(a_j^{(l)})$. For tanh for instance, $h'(a) = 1 - \tanh^2(a) = 1 - (z_j^{(l)})^2$, so the activation $z_j^{(l)}$ must have been stored during the forward pass. More generally, pre-activations $a_j^{(l)}$ must be retained in memory, too.

2.4—Correct: B, E.

(A) **False.** This is precisely what the double descent phenomenon contradicts. Beyond the interpolation threshold, test error can decrease again as the number of parameters continues to

grow—highly overparameterised models can generalise well (better) despite perfectly fitting the training data.

(B) True. By randomly dropping units at each forward pass, dropout prevents any single neuron from relying on the presence of specific other neurons. Each unit must instead learn features that are individually informative, producing more robust and distributed representations.

(C) False. Dropout is disabled at inference time. Applying it during inference would introduce randomness into predictions, making the model non-deterministic and degrading performance without any regularisation benefit — there is no overfitting to prevent at inference.

(D) False. Inductive biases constrain the hypothesis space based on assumptions about the solution—but those assumptions can be wrong. If the prior encoded by a regulariser does not match the true data distribution, the constraint actively hurts generalisation by excluding the true solution. For example, flipping-based augmentation is harmful for tasks where orientation is discriminative (e.g. reading digits).

(E) True. The L2 penalty $\lambda\|\mathbf{w}\|^2$ adds a gradient term $2\lambda\mathbf{w}$, shrinking weights at every step towards zero. This encodes a prior preference for solutions with small weight magnitudes, corresponding to smoother functions.

2.5—Correct: A, C, D.

(A) True. Each convolutional layer integrates information over a local neighbourhood. Stacking layers compounds this: a unit at layer l has a receptive field that grows with l , eventually spanning large regions of the input. This is why shallow layers detect local features such as edges, while deeper layers detect higher-level structures such as textures, parts, and objects.

(B) False. A convolutional layer has $(C_{in}M^2 + 1)C_{out}$ learnable parameters. A max pooling layer with the same window size M , stride, and padding has zero learnable parameters—it is a fixed operation that selects the maximum value in each local window, with no weights to learn or update.

(C) True. A 1×1 conv with K filters outputs K channels while preserving $H \times W$; it performs a learned linear combination across the channel dimension at each spatial position independently.

(D) True. In a residual block, the output is $\mathbf{x} + F(\mathbf{x})$. The gradient of the loss w.r.t. $\mathbf{x}$ contains an additive $+1$ term from the identity branch, guaranteeing a non-zero gradient path to earlier layers regardless of depth.

(E) False. CNNs are translation-*equivariant*, not translation-invariant. Equivariance means that shifting the input by a vector $\mathbf{d}$ shifts each feature map by the same $\mathbf{d}$ —the pattern of activations moves with the input. Invariance would mean the final output (e.g. the class scores) is completely unchanged by the shift. A CNN with a fully connected classifier is therefore not translation-invariant: shifting the input changes the spatial position of features and thus the activations fed into the classifier.

2.6—Correct: B, E.

(A) False. The attention matrix $\text{softmax}(QK^\top/\sqrt{D_k})$ has shape $N \times N$, requiring $O(N^2)$ memory (quadratic dependence)—a key bottleneck for long sequences.

(B) True. If the same matrix U were used for both queries and keys, the pre-softmax scores would be $XUU^\top X^\top$, which is symmetric. This forces $a_{nm} = a_{mn}$, preventing the model from learning asymmetric relationships. Separate $W^{(q)}$ and $W^{(k)}$ break this symmetry.

(C) False. With H heads each projecting to dimension D/H , the per-head parameter count is $3 \times (D \times D/H)$. Summing over H heads gives $3D^2$ —identical to a single head with dimension D . The number of parameters is the same, not H times larger.

(D) False. The preference for layer normalisation over batch normalisation in transformers is not primarily about computational cost. Both have comparable complexity. The key reasons are: (1) layer norm normalises each token’s representation independently across its D features, making it independent of batch size and sequence length; (2) batch norm normalises across the batch dimension, which is problematic for variable-length sequences where padding would corrupt batch statistics; and (3) batch norm behaves differently at training and inference time (requiring running averages), whereas layer norm is identical in both phases.

(E) True. If the components of q and k are i.i.d. with mean 0 and variance 1, the dot product $q \cdot k = \sum_{i=1}^{D_k} q_i k_i$ has variance D_k . For large D_k , large dot products push the softmax into its saturation region, producing near-one-hot attention weights and near-zero gradients. Dividing by $\sqrt{D_k}$ restores unit variance.

Part 3—Short Answers

Note: these are just examples of answers, others may be accepted.

3.1—Vanishing gradients

(a) Backpropagation computes gradients via the chain rule:

$$\frac{\partial \mathcal{L}}{\partial w^{(k)}} \propto \prod_{l=k+1}^L h'(a^{(l)}) w^{(l)}$$

If $|h'(a^{(l)}) w^{(l)}|$ is less than 1, this product shrinks exponentially with depth. Gradients reaching the earliest layers are vanishingly small, so those layers update extremely slowly or not at all. Since early layers learn low-level features that later layers build on, the entire representation hierarchy fails to train effectively.

(b) The sigmoid derivative

$$\sigma'(a) = \sigma(a)(1 - \sigma(a)) = \frac{e^{-a}}{(1 + e^{-a})^2}$$

has a maximum in 0 and decays rapidly to zero as $|a|$ increases (saturation). The tanh derivative $\tanh'(a) = 1 - \tanh^2(a)$ saturates similarly, since $|\tanh(a)| \rightarrow 1$ for $a \rightarrow \pm\infty$. Both activations systematically shrink gradients in deep networks.

(c) A residual (skip) connection addresses this by adding the input of a block directly to its output:

$$\mathbf{z}^{(l)} = F(\mathbf{z}^{(l-1)}) + \mathbf{z}^{(l-1)}$$

where $F(\cdot)$ is the transformation learned by the block. Rather than learning the full desired mapping, the block only needs to learn the *residual*—the correction on top of the identity. The gradient flow benefit follows directly from the chain rule: the gradient of the loss with respect to $\mathbf{z}^{(l-1)}$ contains an additive +1 term from the identity branch, which guarantees a direct path for gradients to flow back to earlier layers regardless of what F does. Even if $\partial F / \partial \mathbf{z}^{(l-1)}$ vanishes, the gradient still propagates unchanged through the skip connection. Introduced by He et al. (2015) for CNNs (ResNets), this design enabled training networks with hundreds of layers and is now ubiquitous—transformers also rely on residual connections around every attention and feed-forward sub-block.

3.2—Catastrophic forgetting

(a) The limitation is *catastrophic forgetting*: when a neural network is trained sequentially on a new task, gradient descent modifies the shared weights to minimise the new loss, overwriting the configurations that encoded prior knowledge. Unlike biological memory, which is consolidated gradually and selectively, standard gradient descent has no mechanism to protect weights that were important for previous tasks — it treats all parameters as equally available for modification. *Continual learning* is the paradigm that studies how to train models sequentially without catastrophic forgetting. One approach is the *experience replay*, which consists in maintaining a buffer of past examples interleaved with new training data, preventing overwriting by keeping the network exposed to old tasks.

(b) Transfer learning leverages knowledge from a *source task* with abundant data to improve learning on a different but related *target task* with scarce data. The two main strategies are: *feature extraction*—freeze the pretrained weights and retrain only the final classifier; and *fine-tuning*—initialise with pretrained weights and retrain the whole network with a small learning rate. Example from the course: Esteva et al. (Nature, 2017) fine-tuned a CNN pretrained on ImageNet to classify skin lesions, achieving dermatologist-level accuracy. This works because early CNN layers learn generic low-level features (edges, textures) useful across many visual tasks, while later layers encode task-specific abstractions that need to be adapted.

(c) AI systems learn statistical co-occurrence patterns without any direct sensorimotor experience of heat or causation. Human infants develop intuitive physical knowledge through embodied interaction with their environment, not passive observation of data. One research direction attempting to close this gap is *world models*: rather than learning from curated datasets, a model learns an internal predictive representation of the world from raw sensory experience, allowing it to anticipate the consequences of actions. A concrete example from the course is JEPA (Assran et al., 2025), which learns by predicting abstract representations of future sensory states rather than raw pixels.

Part 4—Exercise

4.1—CNN architecture

(a)

Layer	Output dimensions	# weight params	# bias params
INPUT	(28, 28, 1)	—	—

Layer	Output dimensions	# weight params	# bias params
CONV-1	(24, 24, 32)	$1 \times 5^2 \times 32 = 800$	32
POOL-1	(8, 8, 32)	0	0
CONV-2	(4, 4, 64)	$32 \times 5^2 \times 64 = 51,200$	64
POOL-2	(2, 2, 64)	0	0
FLATTEN	(256,)	0	0
FC-1	(200,)	$256 \times 200 = 51,200$	200
FC-2	(10,)	$200 \times 10 = 2,000$	10

Output size formula for convolution/pooling with kernel size M , no padding and stride S : $H_{out} = \lfloor (H_{in} - M)/S \rfloor + 1$.

The total number of parameters per convolutional layer is $(C_{in}M^2 + 1)C_{out}$, where the +1 accounts for the bias per filter. For FC layers: $(D_{in} + 1)D_{out}$.

- CONV-1: $H_{out} = (28 - 5)/1 + 1 = 24$. Tot. params: $(1 \times 5^2 + 1) \times 32 = 26 \times 32 = 832$.
- POOL-1: $H_{out} = (24 - 3)/3 + 1 = 8$. No parameters.
- CONV-2: $H_{out} = (8 - 5)/1 + 1 = 4$. Tot. params: $(32 \times 5^2 + 1) \times 64 = 801 \times 64 = 51,264$.
- POOL-2: $H_{out} = (4 - 2)/2 + 1 = 2$. No parameters.
- FLATTEN: $2 \times 2 \times 64 = 256$.
- FC-1: Tot. params: $(256 + 1) \times 200 = 51,400$.
-

FC-2: Tot. params: $(200 + 1) \times 10 = 2,010$.

(b) Total trainable parameters:

$$832 + 51,264 + 51,400 + 2,010 = \mathbf{105,506}$$

CONV-2 and FC-1 contribute almost equally, each with approx. 51k parameters—together accounting for over 97% of all parameters. This is notable: in typical LeNet-style architectures the fully connected layers dominate, but here the second convolutional layer is equally expensive because it operates on 32 input channels with a 5×5 kernel and produces 64 output channels. The large parameter count in CONV-2 motivates using smaller kernels (e.g. 3×3) in deeper architectures such as VGG.

(c) With 32×32 input:

- CONV-1: $H_{out} = (32 - 5)/1 + 1 = 28$. Output: (28, 28, 32).
- POOL-1: $H_{out} = (28 - 3)/3 + 1 = 9.67 \rightarrow 9$. Output: (9, 9, 32).
- CONV-2: $H_{out} = (9 - 5)/1 + 1 = 5$. Output: (5, 5, 64).
- POOL-2: $H_{out} = (5 - 2)/2 + 1 = 2.5 \rightarrow 2$. Output: (2, 2, 64).
- FLATTEN: $2 \times 2 \times 64 = 256$.

The FLATTEN dimension is the same (256), so FC-1 would also have $256 \times 200 = 51,200$ weight parameters — identical to the 28×28 case. This illustrates that the pooling layers absorb the extra spatial resolution, leaving the same flattened representation.

4.2—Automatic Differentiation

(a)

$$v_1 = x, \quad v_2 = w, \quad v_3 = b, \quad v_4 = t$$

$$v_5 = v_1 \times v_2 = xw$$

$$v_6 = v_5 + v_3 = xw + b$$

$$v_7 = \text{ReLU}(v_6) = y$$

$$v_8 = v_7 - v_4 = y - t$$

$$v_9 = v_8^2 = (y - t)^2 = \mathcal{L}$$

(b) Forward pass:

$$v_1 = 2, \quad v_2 = 3, \quad v_3 = -7, \quad v_4 = 1$$

$$v_5 = 2 \times 3 = 6$$

$$v_6 = 6 + (-7) = -1$$

$$v_7 = \text{ReLU}(-1) = 0$$

$$v_8 = 0 - 1 = -1$$

$$v_9 = (-1)^2 = 1$$

(c) Backward pass, starting from $\partial\mathcal{L}/\partial v_9 = 1$ and applying

$$\bar{v}_i = \frac{\partial\mathcal{L}}{\partial v_i} = \sum_{j \in \text{ch}(i)} \bar{v}_j \frac{\partial v_j}{\partial v_i}$$

To evaluate the adjoint variables, which is the derivative of the loss with respect to the corresponding quantity.

$$\bar{v}_8 = \bar{v}_9 \frac{\partial v_9}{\partial v_8} = \bar{v}_9 \times 2v_8 = -2$$

$$\bar{v}_7 = \bar{v}_8 \frac{\partial v_8}{\partial v_7} = \bar{v}_8 \times 1 = -2$$

$$\bar{v}_6 = \bar{v}_7 \frac{\partial v_7}{\partial v_6} = \bar{v}_7 \times \begin{cases} 1 & \text{if } v_6 > 0 \\ 0 & \text{if } v_6 < 0 \end{cases} = 0$$

$$\bar{v}_5 = \bar{v}_6 \frac{\partial v_6}{\partial v_5} = \bar{v}_6 \times 1 = 0$$

$$\bar{v}_4 = \bar{v}_8 \frac{\partial v_8}{\partial v_4} = \bar{v}_8 \times -1 = 2$$

$$\bar{v}_3 = \bar{v}_6 \frac{\partial v_6}{\partial v_3} = \bar{v}_6 \times 1 = 0$$

$$\bar{v}_2 = \bar{v}_5 \frac{\partial v_5}{\partial v_2} = \bar{v}_5 \times v_1 = 0$$

$$\bar{v}_1 = \bar{v}_5 \frac{\partial v_5}{\partial v_1} = \bar{v}_5 \times v_2 = 0$$

All of the gradients requested

$$\bar{v}_6 = \frac{\partial \mathcal{L}}{\partial (wx + b)}, \quad \bar{v}_3 = \frac{\partial \mathcal{L}}{\partial b}, \quad \bar{v}_2 = \frac{\partial \mathcal{L}}{\partial w}.$$

are zero despite $\mathcal{L} = 1 \neq 0$. This is the **dead ReLU** phenomenon: because $v_6 = wx + b = -1 < 0$, the ReLU gate is closed and its local gradient $1[v_6 > 0] = 0$. By the chain rule, this zero propagates back through the entire graph, so neither w nor b receives any gradient signal and neither parameter is updated. The practical consequence is severe: if a unit remains inactive across all training examples, it will never recover—it becomes a permanently dead neuron, contributing nothing to the network’s output. This motivates careful weight initialisation (e.g. He initialisation) and the use of modified versions of the ReLU activation functions—for instance, Leaky ReLU, which passes a small gradient even for negative inputs.